

```

from netmiko import ConnectHandler

# Ip adres ontvangen van de DHCP
host = input("Enter the DHCP-assigned IP of the switch: ")

# Aruba CX aanmeld gegevens
aruba_cx = {
    "device_type": "aruba_aoscx",
    "host": host,
    "username": "admin",
    "password": "3C0mIRF",
}

# leest de VLANs van de VLANs.txt die aangepast kan worden
def read_vlans_from_file(filename="Vlans.txt"):
    vlan_dict = {}
    try:
        with open(filename, "r") as file:
            for line in file:
                parts = line.strip().split(",")
                if len(parts) >= 2:
                    vlan_id, vlan_name = parts[:2]
                    vlan_dict[vlan_id.strip()] = vlan_name.strip()
    except FileNotFoundError:
        print(f"Error: {filename} not found. Please make sure the file exists in the same directory.")
    except Exception as e:
        print(f"Error reading VLANs from {filename}: {e}")
    return vlan_dict

# vraagt de gebruikers input van welke vlans gebruikt worden
def get_vlans_from_user():
    vlan_list = []
    while True:
        vlan_input = input("Choose VLAN (enter 0 to finish): ").strip()
        if vlan_input == "0":
            break
        if vlan_input.isdigit():
            vlan_list.append(vlan_input)
        else:
            print("Invalid VLAN number. Please enter only digits.")
    return vlan_list

# generen van de range door "Member" input vraag
def generate_interface_range(stack_size):
    interface_ranges = [f"{i}/1/1-{i}/1/48" for i in range(1, stack_size + 1)]
    return ",".join(interface_ranges)

try:
    print("\nConnecting to the Aruba CX switch...")
    net_connect = ConnectHandler(**aruba_cx)
    print("Connection successful!")

    while True:
        try:
            stack_size = int(input("Enter the number of required stack members (1-6): "))
            if 1 <= stack_size <= 6:
                break
            print(f"Invalid stack size entered: {stack_size}")
            print("Invalid input. Please enter a number between 1 and 6.")

```

```

except ValueError:
    print("Invalid stack size input (non-integer).")
    print("Invalid input. Please enter a valid number.")

interface_range = generate_interface_range(stack_size)
hostname = input("Enter new hostname: ")
mgmt_ip = input("Enter IP address for VLAN 3200: ")
mgmt_mask = input("Enter subnet mask for VLAN 3200: ")
Location = input("Location of the Switch : ")
vlan_dict = read_vlans_from_file("Vlans.txt")

    # verwijderd de extra members door vraag "required stack members"
print("\nRemoving unnecessary stack members...")

for i in range(6, stack_size, -1):
    print(f"removing VSF member {i}...")

    # stuur commando voor conf t
    net_connect.config_mode()

    # stuurt de command om Members te verwijderen + de input van de user
    output = net_connect.send_command_timing(f"no vsf member {i}")

    # controleert de welke output komt en vergelijkt
    if "Do you want to continue" in output:
        confirmation = net_connect.send_command_timing("y")

    net_connect.exit_config_mode()

# Configureer VLAN 3200 voor management + locatie
print("\nConfiguring Management VLAN (3200)...")
net_connect.send_config_set([
    "vlan 3200",
    "name Management",
    "interface vlan 3200",
    f"ip address {mgmt_ip} {mgmt_mask}",
    "no shutdown",
    "exit",
    f"snmp-server system-location {Location}",
    f"snmp-server system-description {Location}",
    f"ip source-interface radius {mgmt_ip}",
])

# Vraag VLANs op die geconfigureerd moeten worden

print("\nEnter VLAN numbers to add (enter 0 to stop):")
selected_vlans = get_vlans_from_user()
blackhole_vlans = {"666", "1666", "2666", "3666"}
blackhole_vlan = None

if selected_vlans:
    print("\nAdding VLANs...")
    for vlan_id in selected_vlans:
        vlan_name = vlan_dict.get(vlan_id, f"VLAN{vlan_id}")
        if vlan_id in blackhole_vlans:
            blackhole_vlan = vlan_id
            print(f"Detected Blackhole VLAN: {blackhole_vlan}")
        net_connect.send_config_set([
            f"vlan {vlan_id}",
            f"name {vlan_name}"

```

```

    ])
# Indien geen blackhole VLAN gedetecteerd werd, vraag het op aan de gebruiker
if not blackhole_vlan:
    while True:
        blackhole_vlan = input("Enter the Blackhole VLAN (666, 1666, 2666, or 3666): ").strip()
        if blackhole_vlan in blackhole_vlans:
            net_connect.send_config_set([
                f"vlan {blackhole_vlan}",
                "name Blackhole"
            ])
            break
        else:
            print("Invalid Blackhole VLAN. Please choose from 666, 1666, 2666, or 3666.")

# plaats alle interfaces toe aan de blackhole VLAN
print(f"\nAssigning all interfaces to Blackhole VLAN {blackhole_vlan}...")
net_connect.send_config_set([
    f"interface {interface_range}",
    "no shutdown",
    f"vlan access {blackhole_vlan}",
    "no power-over-ethernet",
    "exit"
])
# Configureer trunk voor LAG1 met alle nodige VLANs
allowed_vlans = ",".join(selected_vlans + ["1", "3200", blackhole_vlan])
print(f"\nConfiguring LAG1 to allow VLANs: {allowed_vlans}...")
net_connect.send_config_set([
    "interface lag 1",
    "no shutdown",
    "vlan trunk native 1",
    f"vlan trunk allowed {allowed_vlans}"
])
# Activeer client tracking op de geselecteerde VLANs
print("\nAdding Client track IP to VLANs...")
net_connect.send_config_set([
    f"vlan {' '.join(map(str, selected_vlans))}",
    "client track ip",
    "exit"
])

# hostname van de switch in
print("\nSetting hostname...")
net_connect.send_config_set(["configure terminal", f"hostname {hostname}", "exit"])

# configuratie weg naar geheugen
print("\nWriting meme for pietje \U0001F609...")
net_connect.send_command("write memory")

# toont de configuratie
print("\nVerifying configuration...")
running_config = net_connect.send_command("show running-config")
print(running_config)

# sluit de connectie
net_connect.disconnect()
print("\nConfiguration complete. Connection closed.")

except Exception as e:
    print(f"\nERROR: {e}")

```